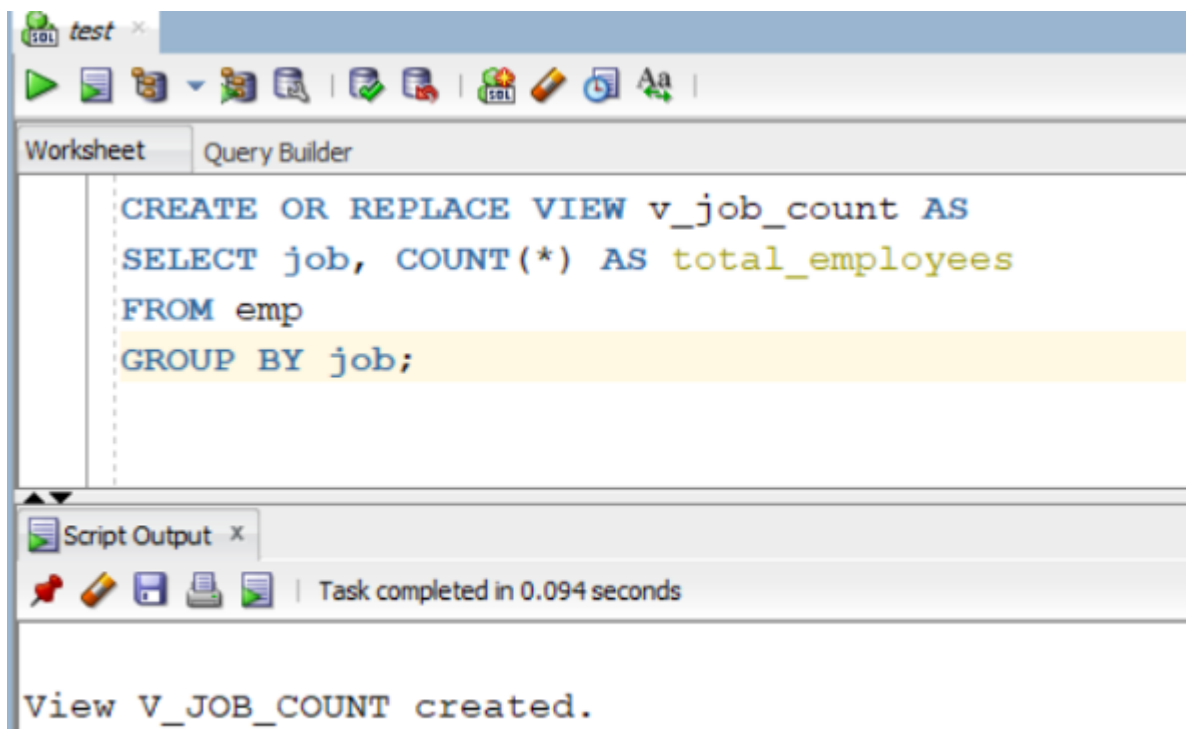


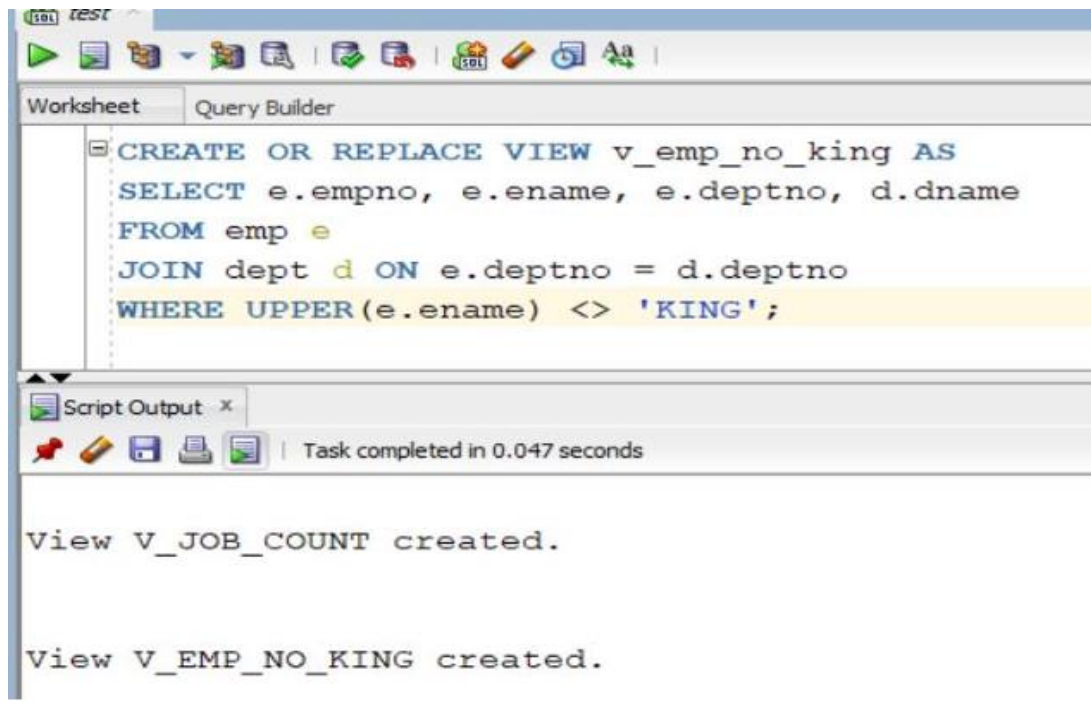
DataBase Lab 10 (POST-Lab Task)

Muhammad Ghulam Mujtaba Qureshi
24k-0535

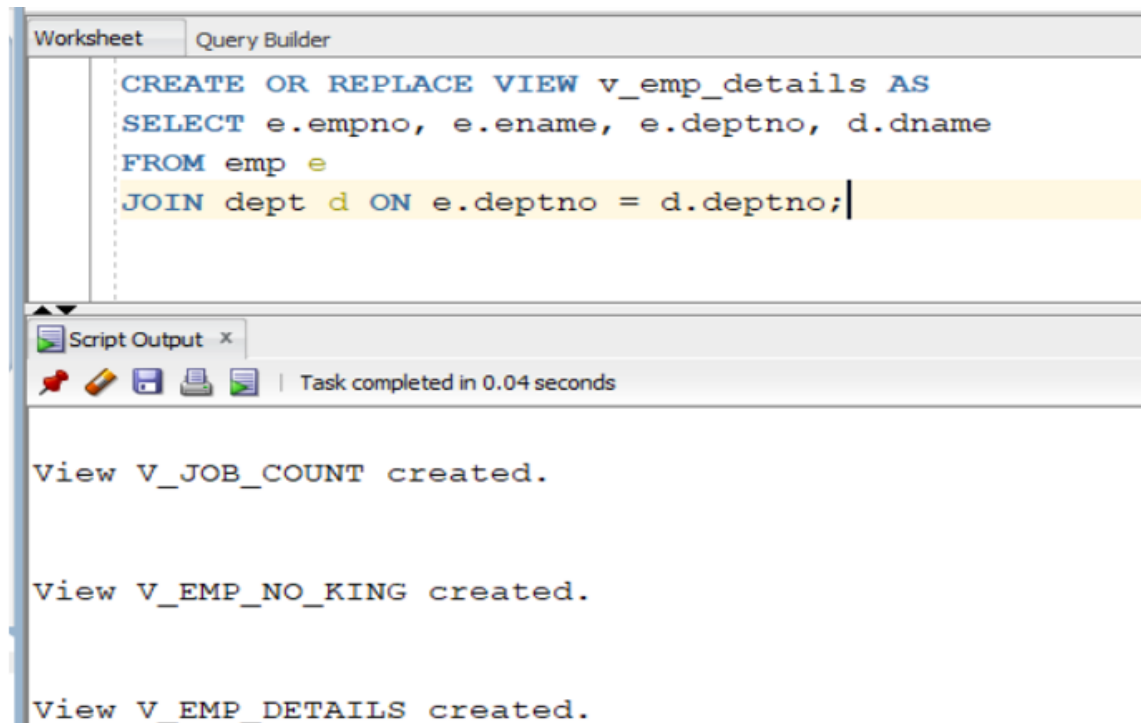
Task 1(a)



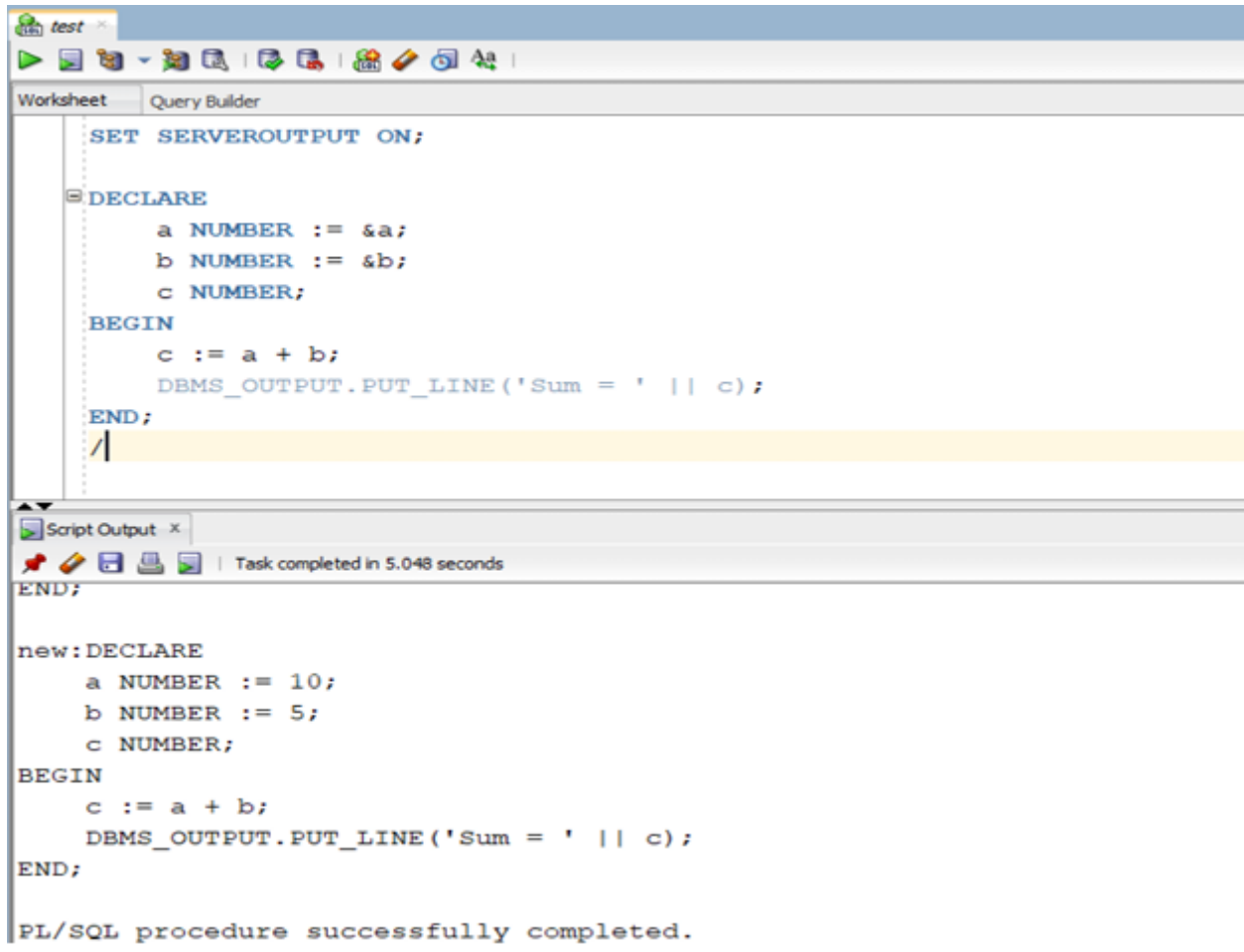
Task 1(b)



Task 1(c)



Task 2



The screenshot displays the Oracle SQL Developer environment. The top window, titled 'test', is the 'Query Builder' tab, showing a PL/SQL script. The script starts with 'SET SERVEROUTPUT ON;', followed by a 'DECLARE' block with three variables: 'a NUMBER := &a;', 'b NUMBER := &b;', and 'c NUMBER;'. The 'BEGIN' block contains 'c := a + b;', 'DBMS_OUTPUT.PUT_LINE('Sum = ' || c);', and ends with 'END;'. The bottom window, titled 'Script Output', shows the execution results. It indicates 'Task completed in 5.048 seconds' and displays the same PL/SQL script with the variable 'a' substituted with the value '10' and 'b' with '5'. The final output line states 'PL/SQL procedure successfully completed.'

```
SET SERVEROUTPUT ON;

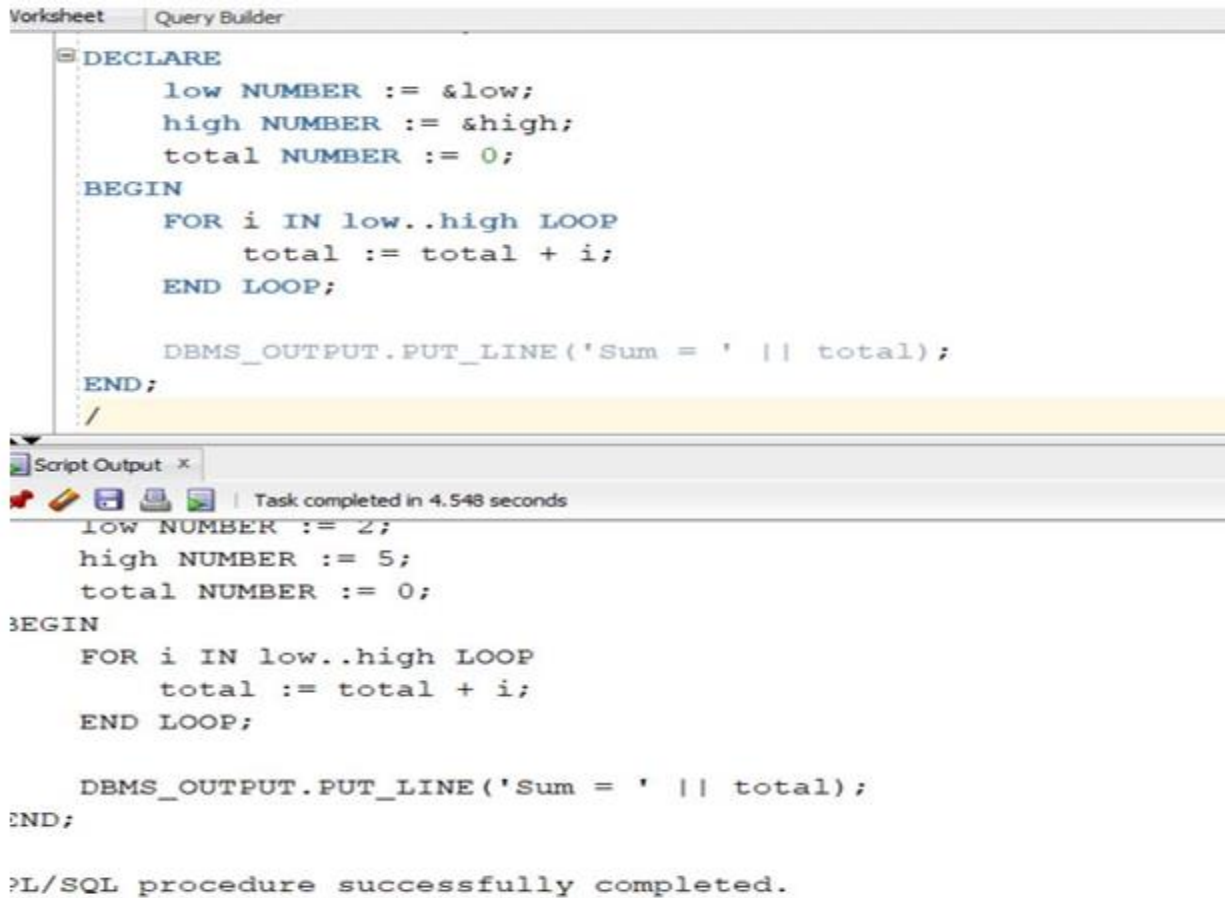
DECLARE
  a NUMBER := &a;
  b NUMBER := &b;
  c NUMBER;
BEGIN
  c := a + b;
  DBMS_OUTPUT.PUT_LINE('Sum = ' || c);
END;
```

Task completed in 5.048 seconds

```
new:DECLARE
  a NUMBER := 10;
  b NUMBER := 5;
  c NUMBER;
BEGIN
  c := a + b;
  DBMS_OUTPUT.PUT_LINE('Sum = ' || c);
END;

PL/SQL procedure successfully completed.
```

Task 3



The screenshot shows the Oracle SQL Developer interface. The top pane, titled 'Worksheet', contains a PL/SQL script. The script declares three variables: 'low' (NUMBER), 'high' (NUMBER), and 'total' (NUMBER), all initialized to 0. It then enters a 'BEGIN' block with a 'FOR' loop from 'low' to 'high', incrementing 'total' by 'i' in each iteration. After the loop, it uses 'DBMS_OUTPUT.PUT_LINE' to display the sum. The script ends with 'END;' and a forward slash. The bottom pane, titled 'Script Output', shows the execution results. It repeats the script code and indicates that the task completed in 4.548 seconds. The final output line states 'PL/SQL procedure successfully completed.'

```
Worksheet | Query Builder

DECLARE
    low NUMBER := &low;
    high NUMBER := &high;
    total NUMBER := 0;
BEGIN
    FOR i IN low..high LOOP
        total := total + i;
    END LOOP;

    DBMS_OUTPUT.PUT_LINE('Sum = ' || total);
END;
/

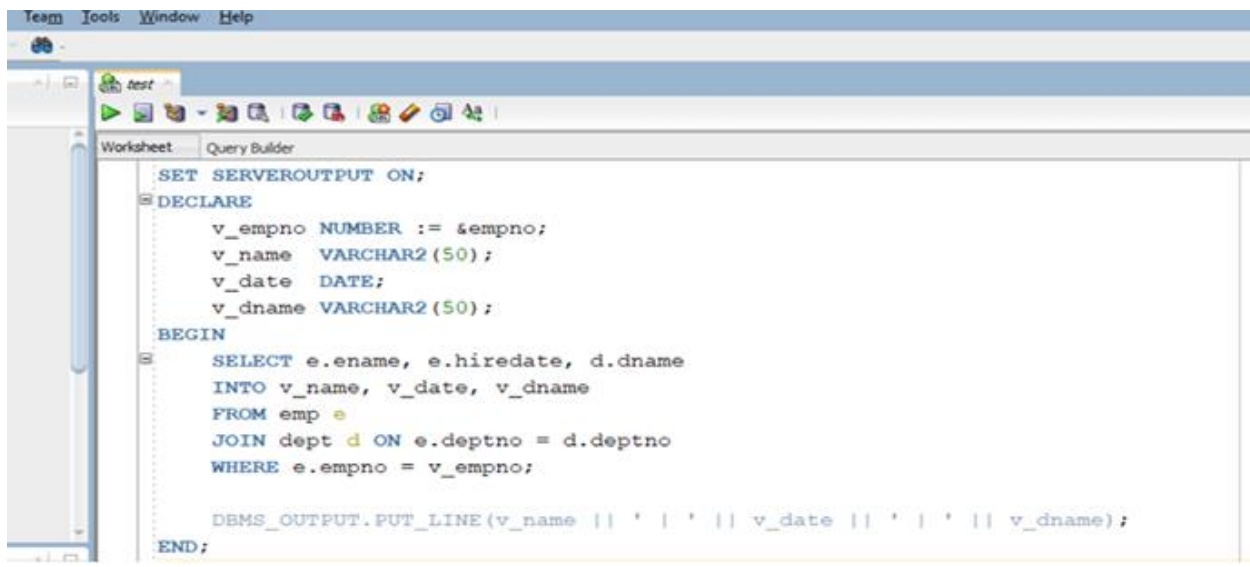
Script Output x
Task completed in 4.548 seconds

low NUMBER := 2;
high NUMBER := 5;
total NUMBER := 0;
BEGIN
    FOR i IN low..high LOOP
        total := total + i;
    END LOOP;

    DBMS_OUTPUT.PUT_LINE('Sum = ' || total);
END;

PL/SQL procedure successfully completed.
```

Task 4



The screenshot shows the Oracle SQL Developer interface. The top pane, titled 'Worksheet', contains a PL/SQL script. It starts with 'SET SERVEROUTPUT ON;'. The script declares four variables: 'v_empno' (NUMBER), 'v_name' (VARCHAR2(50)), 'v_date' (DATE), and 'v_dname' (VARCHAR2(50)). It then enters a 'BEGIN' block with a 'SELECT' statement that joins the 'emp' table with the 'dept' table on 'deptno'. The columns selected are 'e.ename', 'e.hiredate', and 'd.dname', which are stored in the variables 'v_name', 'v_date', and 'v_dname' respectively. The script uses 'DBMS_OUTPUT.PUT_LINE' to display the concatenated values of these variables. The script ends with 'END;'.

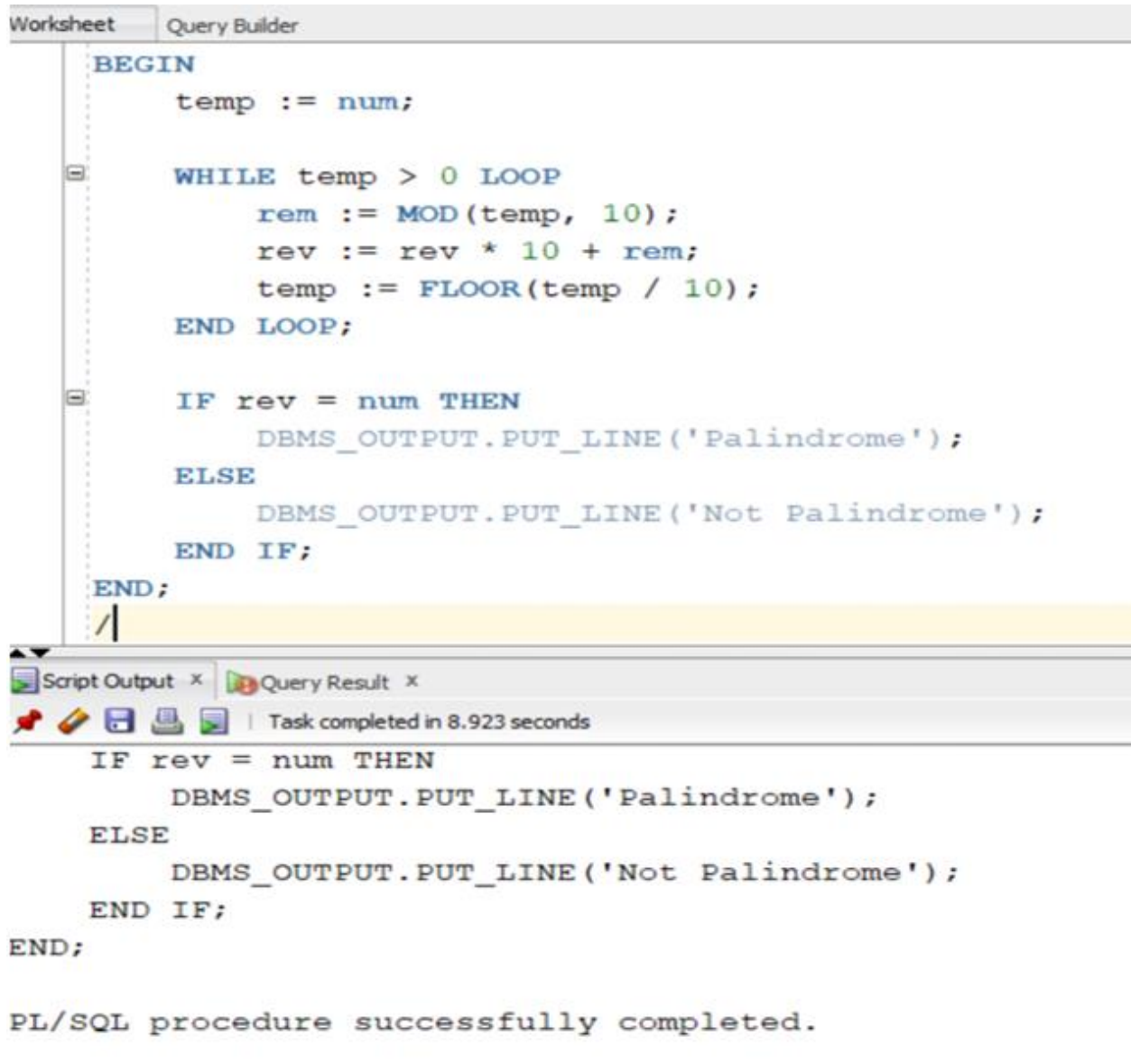
```
Team | Tools | Window | Help

Worksheet | Query Builder

SET SERVEROUTPUT ON;
DECLARE
    v_empno NUMBER := &empno;
    v_name VARCHAR2(50);
    v_date DATE;
    v_dname VARCHAR2(50);
BEGIN
    SELECT e.ename, e.hiredate, d.dname
    INTO v_name, v_date, v_dname
    FROM emp e
    JOIN dept d ON e.deptno = d.deptno
    WHERE e.empno = v_empno;

    DBMS_OUTPUT.PUT_LINE(v_name || ' | ' || v_date || ' | ' || v_dname);
END;
```

Task 5



The screenshot shows a database query builder interface with a 'Worksheet' tab and a 'Query Builder' tab. The main area contains a PL/SQL script for checking if a number is a palindrome. The script uses a WHILE loop to reverse the digits of the number and an IF statement to compare the original number with the reversed number. The output is displayed in a 'Script Output' window, showing the results of the IF statement and a confirmation message.

```
BEGIN
    temp := num;

    WHILE temp > 0 LOOP
        rem := MOD(temp, 10);
        rev := rev * 10 + rem;
        temp := FLOOR(temp / 10);
    END LOOP;

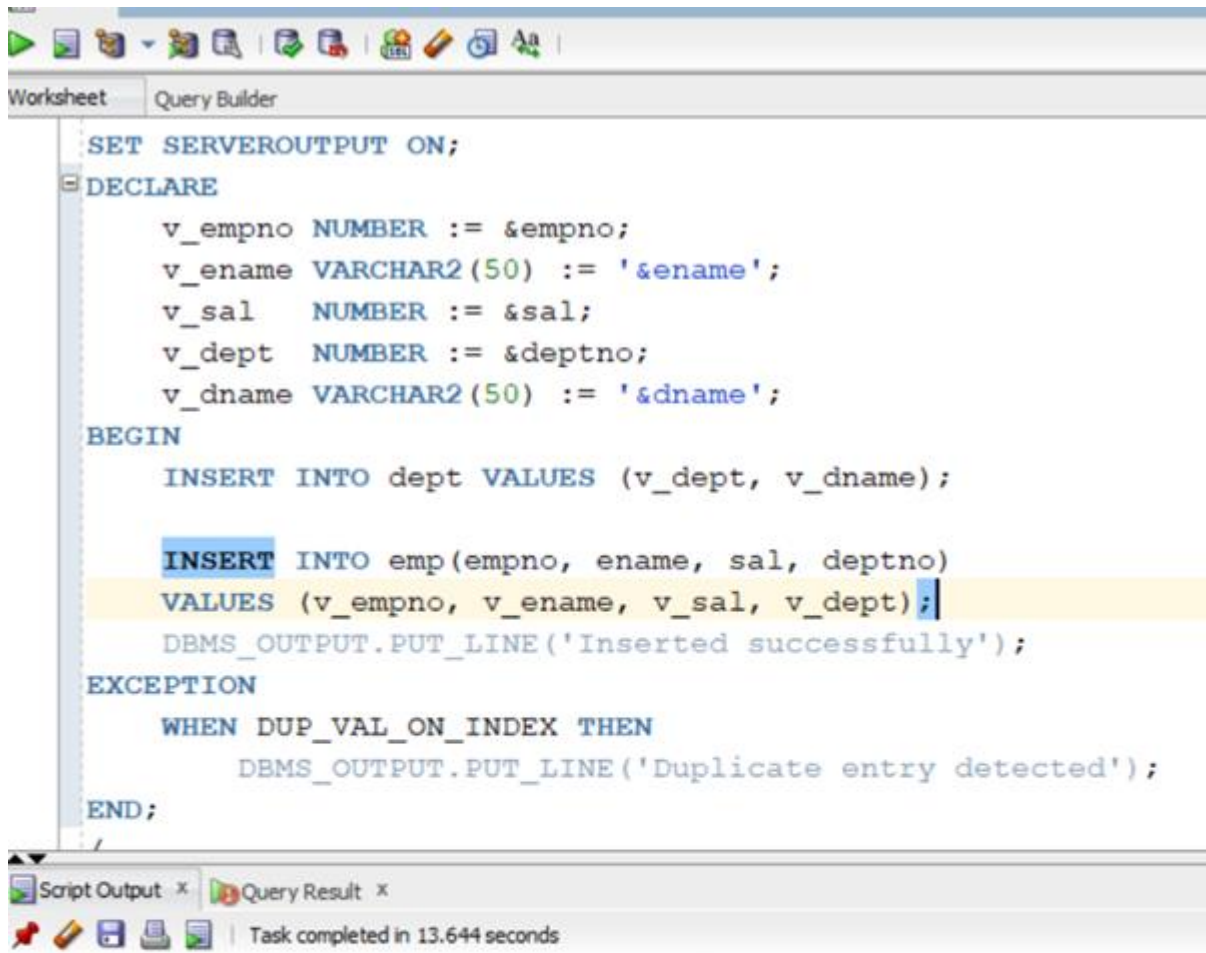
    IF rev = num THEN
        DBMS_OUTPUT.PUT_LINE('Palindrome');
    ELSE
        DBMS_OUTPUT.PUT_LINE('Not Palindrome');
    END IF;
END;
```

Task completed in 8.923 seconds

```
IF rev = num THEN
    DBMS_OUTPUT.PUT_LINE('Palindrome');
ELSE
    DBMS_OUTPUT.PUT_LINE('Not Palindrome');
END IF;
END;
```

PL/SQL procedure successfully completed.

Task 6



```
SET SERVEROUTPUT ON;
DECLARE
    v_empno NUMBER := &empno;
    v_ename VARCHAR2(50) := '&ename';
    v_sal NUMBER := &sal;
    v_dept NUMBER := &deptno;
    v_dname VARCHAR2(50) := '&dname';
BEGIN
    INSERT INTO dept VALUES (v_dept, v_dname);

    INSERT INTO emp(empno, ename, sal, deptno)
    VALUES (v_empno, v_ename, v_sal, v_dept);
    DBMS_OUTPUT.PUT_LINE('Inserted successfully');
EXCEPTION
    WHEN DUP_VAL_ON_INDEX THEN
        DBMS_OUTPUT.PUT_LINE('Duplicate entry detected');
END;
/
```

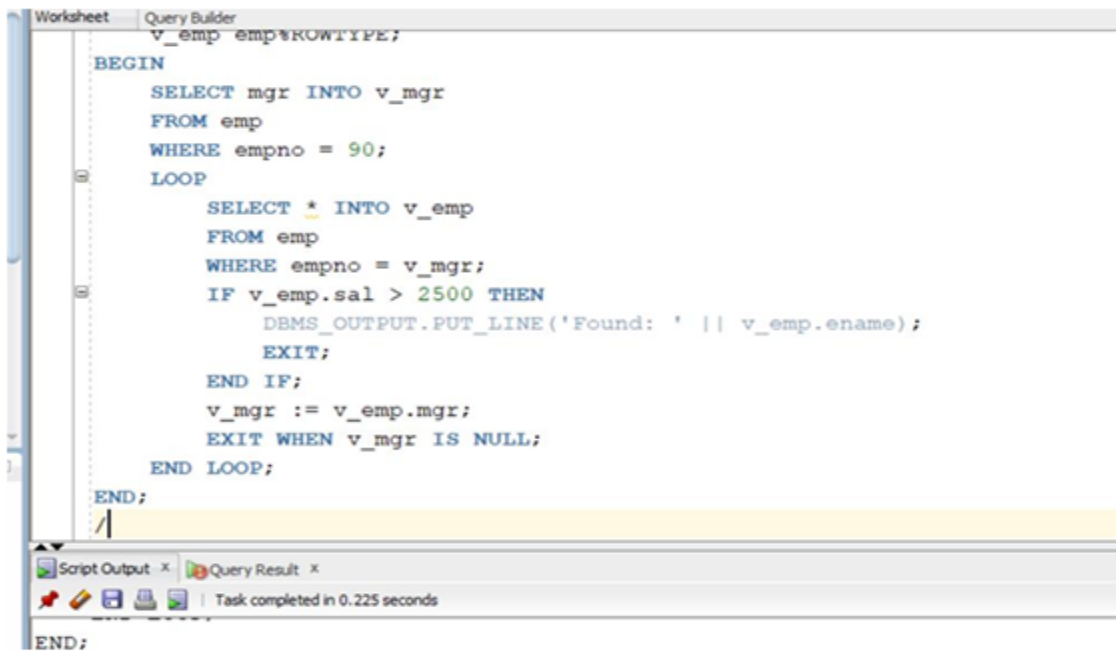
Script Output x Query Result x

Task completed in 13.644 seconds

```
DBMS_OUTPUT.PUT_LINE('Inserted successfully');
EXCEPTION
    WHEN DUP_VAL_ON_INDEX THEN
        DBMS_OUTPUT.PUT_LINE('Duplicate entry detected');
END;
```

PL/SQL procedure successfully completed.

Task 7

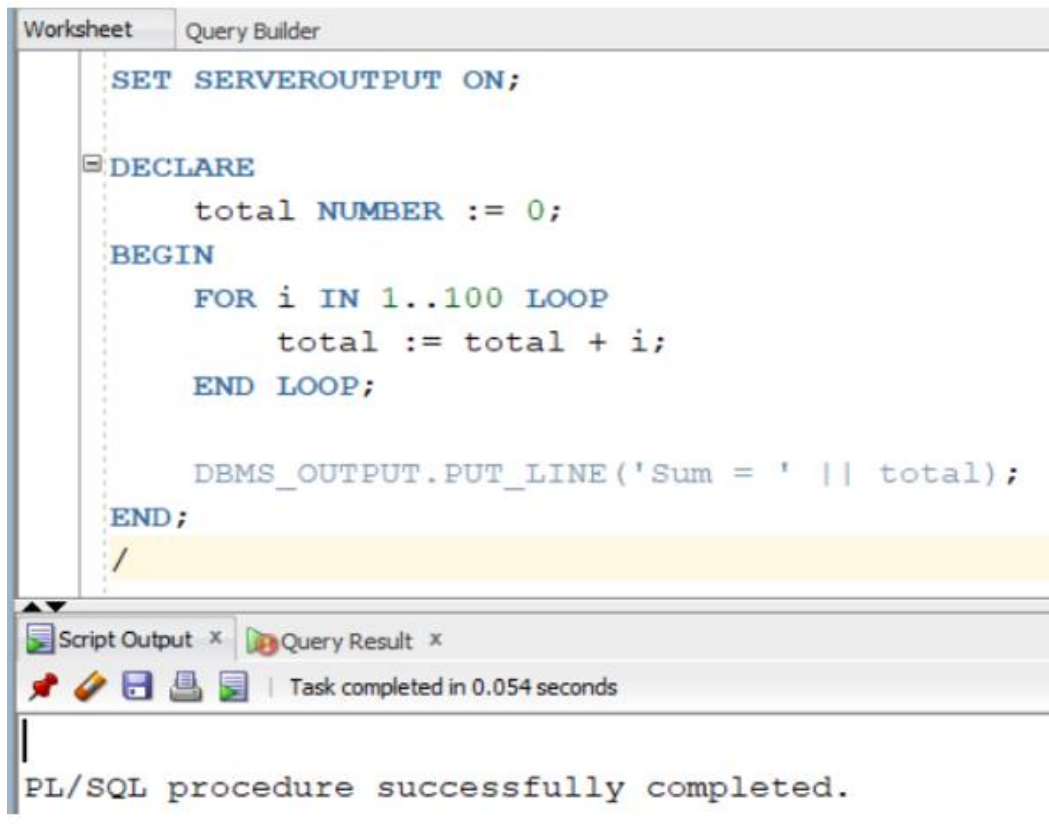


The screenshot shows a SQL IDE with a 'Query Builder' tab. The code is a PL/SQL procedure that finds an employee based on a manager's employee number. It starts by declaring a variable `v_emp emp%ROWTYPE;`. Then it begins a block with `BEGIN`. The first step is to select the manager's details into a variable `v_mgr` from the `emp` table where `empno = 90`. A `LOOP` follows, where it selects all details of the employee whose `empno` is equal to `v_mgr` into `v_emp`. An `IF` statement checks if `v_emp.sal > 2500`. If true, it prints a message using `DBMS_OUTPUT.PUT_LINE('Found: ' || v_emp.ename);` and then `EXIT;`. The `IF` block ends with `END IF;`. The manager variable is updated with `v_mgr := v_emp.mgr;`, and the loop exits when `v_mgr` is null with `EXIT WHEN v_mgr IS NULL;`. The loop ends with `END LOOP;` and the entire block ends with `END;`. The status bar at the bottom indicates 'Task completed in 0.225 seconds'.

```
v_emp emp%ROWTYPE;  
  
BEGIN  
    SELECT mgr INTO v_mgr  
    FROM emp  
    WHERE empno = 90;  
    LOOP  
        SELECT * INTO v_emp  
        FROM emp  
        WHERE empno = v_mgr;  
        IF v_emp.sal > 2500 THEN  
            DBMS_OUTPUT.PUT_LINE('Found: ' || v_emp.ename);  
            EXIT;  
        END IF;  
        v_mgr := v_emp.mgr;  
        EXIT WHEN v_mgr IS NULL;  
    END LOOP;  
END;
```

Script Output x Query Result x
Task completed in 0.225 seconds

Task 8



The screenshot shows a SQL IDE with a 'Query Builder' tab. The code is a PL/SQL procedure that calculates the sum of numbers from 1 to 100. It starts with `SET SERVEROUTPUT ON;`. Then it declares a variable `total NUMBER := 0;`. A `BEGIN` block follows, containing a `FOR` loop from `1..100` where `total := total + i;` is executed for each iteration. The loop ends with `END LOOP;`. After the loop, it prints the result using `DBMS_OUTPUT.PUT_LINE('Sum = ' || total);`. The block ends with `END;`. The status bar at the bottom indicates 'Task completed in 0.054 seconds'. Below the IDE window, the text 'PL/SQL procedure successfully completed.' is displayed.

```
SET SERVEROUTPUT ON;  
  
DECLARE  
    total NUMBER := 0;  
  
BEGIN  
    FOR i IN 1..100 LOOP  
        total := total + i;  
    END LOOP;  
  
    DBMS_OUTPUT.PUT_LINE('Sum = ' || total);  
END;
```

Script Output x Query Result x
Task completed in 0.054 seconds

PL/SQL procedure successfully completed.